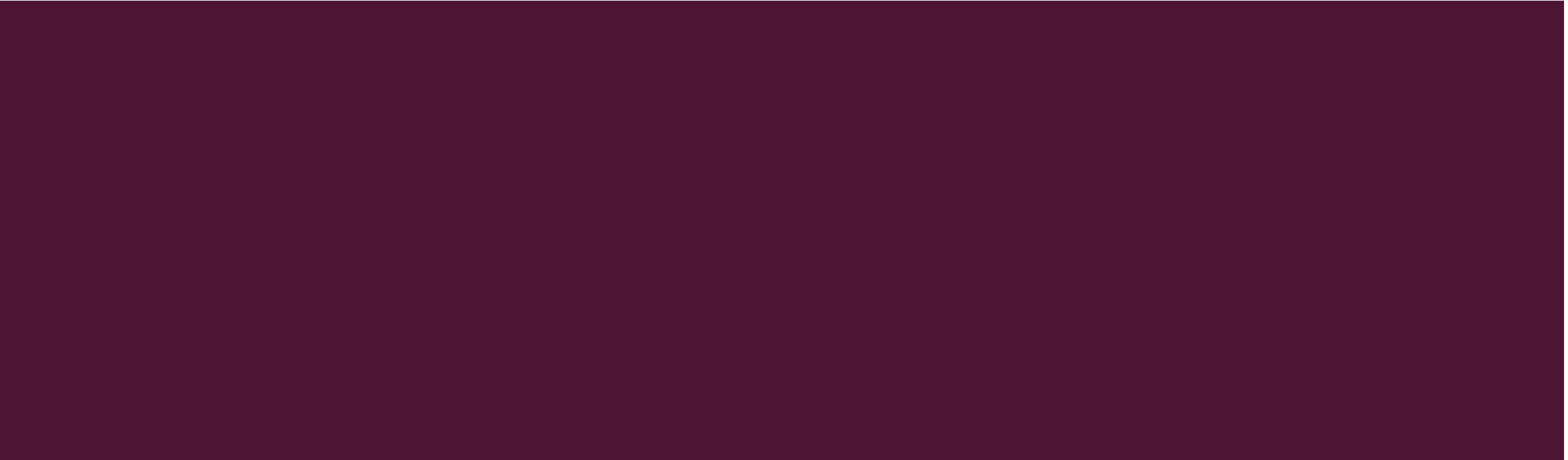




ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 6 – PATRONES DE DISEÑO (II)



ESTRUCTURALES

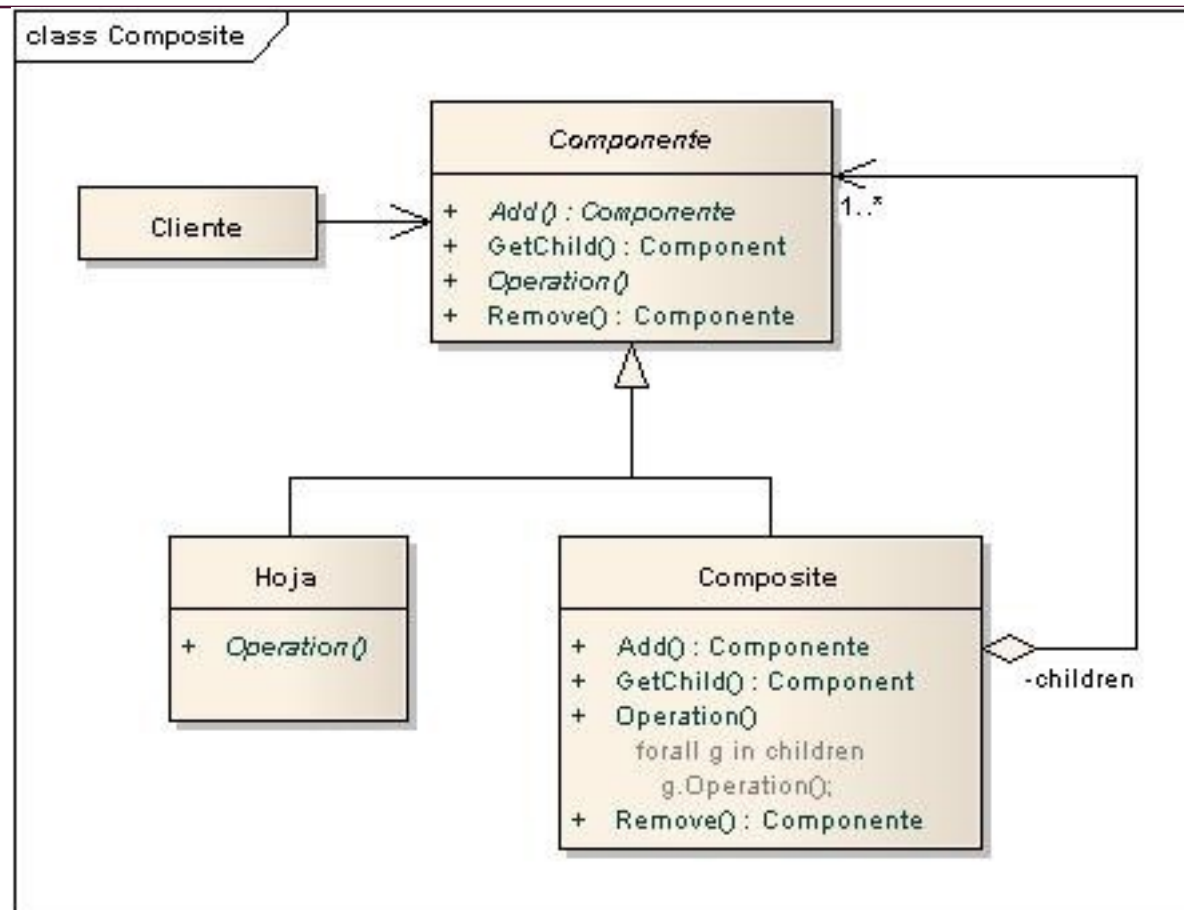
COMPOSITE

Propósito	Compone objetos en estructuras de árbol para representar jerarquías de parte-todo. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.
También conocido como	Compositor
Aplicabilidad	• Se quiere representar jerarquías de objetos parte-todo • Se quiere que los clientes puedan obviar las diferencias entre objetos individuales y composiciones de objetos

ESTRUCTURALES

COMPOSITE

Estructura



ESTRUCTURALES

COMPOSITE

Participantes	Componente Hoja Composite Cliente
Colaboraciones	<i>Cliente</i> usa el interfaz de <i>Componente</i> para interactuar con objetos en la estructura de <i>Composite</i> . Si el receptor es una <i>Hoja</i> , entonces la petición es manejada directamente. Si el receptor es un <i>Composite</i> , lanza la petición a sus hijos.
Consecuencias	• Define jerarquías de clases formadas por objetos primitivos y compuestos • Simplifica el cliente • Facilita añadir nuevos tipos de componentes • Puede hacer que un diseño sea muy general

ESTRUCTURALES

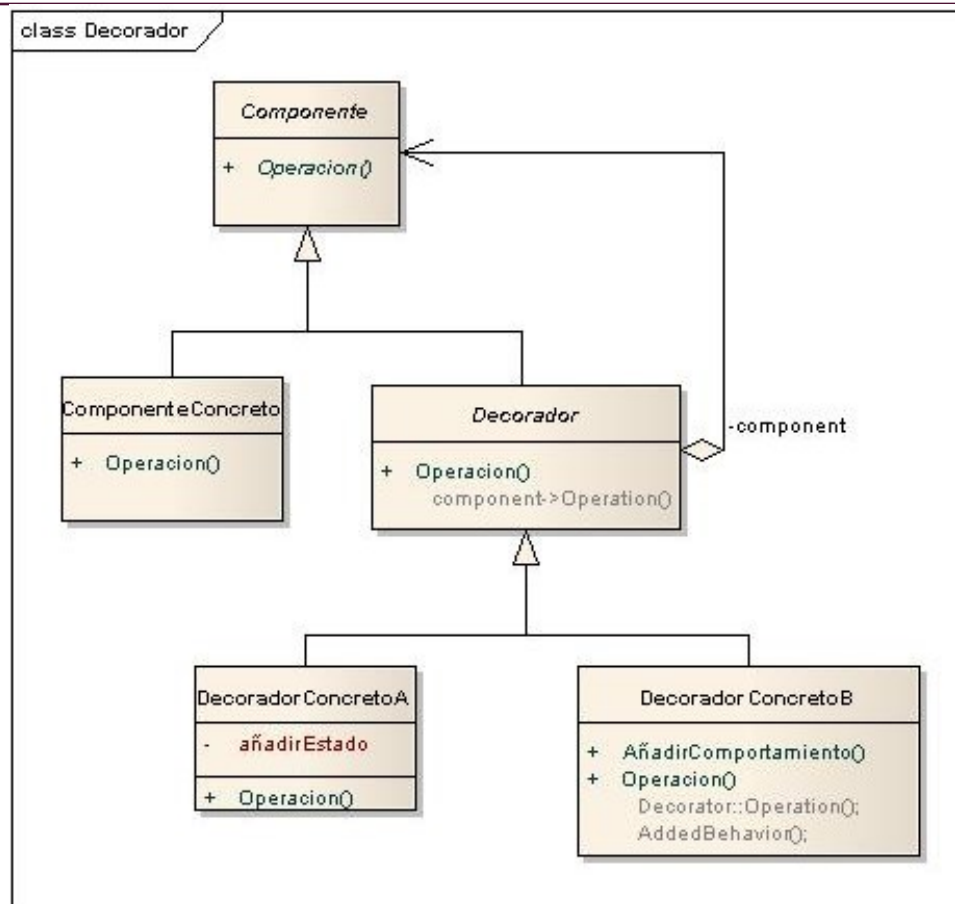
DECORATOR

Propósito	Agrega responsabilidades a un objeto dinámicamente. Es una alternativa flexible a la herencia para añadir funcionalidades.
También conocido como	Wrapper
Motivación	• Se desea añadir funcionalidad a un objeto concreto, no a una clase entera
Aplicabilidad	• Añadir responsabilidades a objetos concretos de manera dinámica y transparente, sin afectar a otros objetos. • Cuando las responsabilidades se pueden otorgar o derogar. • Cuando la herencia sea muy compleja porque implique la creación de múltiples subclases para dar soporte a todas las combinaciones posibles.

ESTRUCTURALES

DECORATOR

Estructura



ESTRUCTURALES

DECORATOR

Participantes	Componente ComponenteConcreto Decorador DecoradorConcreto
Colaboraciones	<i>Decorador</i> adelanta peticiones al objeto <i>Componente</i> . En ocasiones se realizan operaciones previas y posteriores al lanzamiento de la petición
Consecuencias	• Es la más flexible que la herencia estática. Las responsabilidades se añade y eliminan dinámicamente. Facilita definir una propiedad varias veces. • Evita una complejidad en las clases más altas en las jerarquías. Se facilita la definición de nuevos decoradores • Un decorador y el componente al que se refieren no son idénticos (tienen identificador distinto). • Provoca la creación de muchos objetos pequeños parecidos y encadenados, dificultando la depuración.

BIBLIOGRAFÍA

- Gamma, E.; Helm, R.; Johnson, R. & Vlissides, J. M. (1994), *Design Patterns: Elements of Reusable Object-Oriented Software* , Addison-Wesley Professional .



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 6 – PATRONES DE DISEÑO (II)

¿Qué abordaremos en la próxima clase?